

# Dispense Calibration: Histogram Analysis & Auto-Suggested Settings

How we detect miscalibrated dispensers from field data alone, and how the app calculates a corrected Litres Conversion Factor (LCF) and Preload

---

**Prepared for:** Executive Team      **System:** eWater Monitor      **Date:** July 3, 2026

---

## 1. The problem in plain terms

Every dispenser is supposed to release exactly **20 litres** per authorized transaction. In practice, a meter's calibration can drift — mechanically or after a component swap — so it might consistently deliver 16.7 L, or 25 L, for the same "20 L" transaction. There is no direct way to ask the device "are you calibrated?" — we only see the raw pulse counts it reports. This report explains how we infer miscalibration purely by analyzing the pattern of every transaction a meter has logged, and how we translate that pattern into a corrected setting.

## 2. Step 1 — Build a histogram of "should-be-20L" transactions

Every dispense event reports a raw pulse count (the **Flow Count**, or FC) which the device converts to litres using its currently configured Litres Conversion Factor (LCF):  $\text{litres} = \text{FC} \div \text{LCF}$ . We pull every dispense event for a meter over a configurable window (14-30+ days is typical) and compute the litres value for each one, then group them into a histogram restricted to the 10-30 L range — i.e. transactions that are plausibly "one 20 L fill."

Restricting to 10-30 L filters out unrelated activity (leak drips, partial/aborted fills, multi-unit bulk dispenses) so the histogram reflects the population of transactions that were intended to be a standard 20 L fill.

### 3. Step 2 — Smooth the histogram to find the true peak (KDE)

A raw histogram is jagged — the exact "typical fill" value depends on arbitrary bin edges. To get a stable, bin-independent estimate of where transactions actually cluster, we run a standard statistical smoothing technique called **Kernel Density Estimation (KDE)**. Conceptually, instead of stacking transactions into rigid buckets, KDE replaces each transaction with a small smooth "bump" centered on its value, then adds all the bumps together. The result is a smooth curve whose highest point is the best estimate of the meter's typical actual delivery for a nominal 20 L fill.

#### Why not just use the average?

A simple average is distorted by outliers (a handful of 25–29 L bulk fills pulls the average up even though most transactions cluster tightly around one value). The KDE peak instead finds the single most common delivered volume — the mode — which is what a technician would eyeball on a printed histogram, done rigorously and automatically.

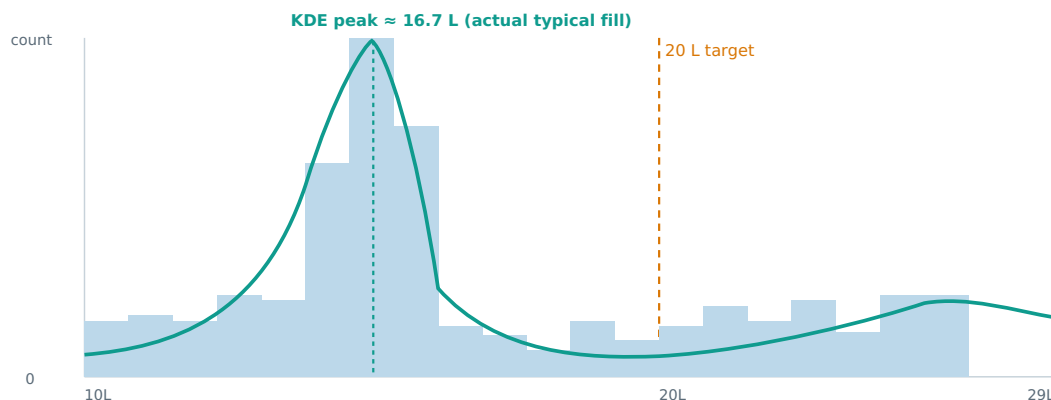
#### Technical detail — bandwidth selection

The "width" of each smooth bump (the bandwidth,  $h$ ) controls how much smoothing is applied — too narrow and the curve is as noisy as the raw histogram; too wide and distinct clusters blur together. We use **Silverman's rule of thumb**, a well-established statistical formula that picks bandwidth automatically from the data's own spread:

$$h = 0.9 \times \min(\text{SD}, \text{IQR} / 1.34) \times n^{-1/5}$$

SD = standard deviation of the sample · IQR = interquartile range ·  $n$  = number of transactions

A minimum of **10 transactions** is required before we compute a peak at all — below that, the sample is too thin to trust, and the app reports "not enough data" rather than guessing.



## 4. Step 3 — Measure the "preload" directly, from a second data source

Litres delivered aren't the whole story. Every dispense includes a small fixed amount of water that flows before the flow meter starts counting pulses — the **preload** (mechanical/valve-open overrun). If we only corrected the LCF, we'd be lumping this fixed offset in with the per-litre conversion, which would make the fix accurate at 20 L but wrong at every other fill size.

We avoid guessing at the preload. eWater dispensers occasionally log a distinct **"no-credit" event** (event type 0x01) — water that flowed without an authorized transaction, e.g. residual drips or a valve-close overrun. These events report their own tick count independent of any billed transaction, which is a direct physical measurement of the meter's unmetered offset. We average this value across every no-credit event seen in the period:

```
measuredPreload = average(unmetered ticks across all no-credit events in the period)
```

No-credit events are sparse — some meters produce several a day, others none for days — so when a meter has zero measured events in the window, the app clearly flags "no measurement — assumed 0" rather than silently treating an absence of data as a confirmed zero.

## 5. Step 4 — Convert the measured peak into a corrected LCF

With a reliable "typical fill" (the KDE peak, in litres) and a directly measured preload (in raw ticks), we can back-solve for the ticks-per-litre value that would have produced exactly 20 L for that same typical transaction:

```
// 1. total raw ticks currently recorded for the typical fill
rawTicks = kdePeak × currentLcf

// 2. remove the fixed offset to isolate ticks caused by
actual flow
meteredTicks = rawTicks - measuredPreload

// 3. the LCF that turns those metered ticks into exactly 20
L
```

```
suggestedLcf = round(meteredTicks / 20)
```

If the result is zero or negative (i.e. the measured preload alone exceeds the raw tick count for a typical fill — a sign of noisy/insufficient data), the app does not present a suggestion at all rather than propose a nonsensical setting.

**Why this generalizes across meter types automatically:** the formula scales directly off each meter's own current LCF, so it self-corrects for meters on very different hardware scales (e.g. older "v3" meters at LCF  $\approx 71$  vs. standard meters at LCF  $\approx 350$ –400) without needing separate rules per meter family.

## 6. Worked examples from live field data

The table below shows three real dispensers from the current fleet, each illustrating a different scenario.

| Asset           | Transactions (14d) | Current LCF | KDE peak (L) | Measured preload          | Suggested LCF | Interpretation                                                                                   |
|-----------------|--------------------|-------------|--------------|---------------------------|---------------|--------------------------------------------------------------------------------------------------|
| 2240            | 481                | 373         | 16.7         | 215.2 ticks (44 events)   | 301           | <b>Over-delivering — meter counts too many ticks per litre; LCF should drop</b>                  |
| 2706 (v3 meter) | 459                | 71          | 20.9         | 17 ticks (1 event)        | 73            | Close to target; small correction. Same formula applies to older v3 hardware                     |
| 662             | 15                 | 365         | 25.0         | none measured (assumed 0) | 456           | Sample is thin (15 tx) and no no-credit events observed — suggestion shown with lower confidence |

**Asset 2240 — before correction**

**16.7 L**

typical delivered volume vs. 20 L target (16.5% short)

**Asset 2240 — after applying LCF 301**

**≈ 20.0 L**

projected typical delivered volume once corrected

## 7. What "Apply" does in the app

Each dispenser's chart page shows this analysis live and, where a valid suggestion exists, an **"Apply suggested settings"** button. This is a deliberate,

human-confirmed action — the app never writes to a device automatically. Clicking it opens a confirmation dialog showing the before/after values for both settings, and only on explicit confirmation does it send the correction to the physical dispenser:

1. **LCF** Litres Conversion is updated to the suggestedLcf value calculated above.
2. **Preload** is set to the measured preload (rounded to the nearest tick), or 0 if none was measured in the window.

Both settings are written via eWater's managed setting-change API; the device applies them on its next check-in. Because both numbers come from the meter's own logged transactions, no site visit, manual test-pour, or firmware change is required to recalibrate a drifting meter.

## 8. Confidence & limitations — talking points for Q&A

- **Sample size matters.** We require at least 10 transactions before showing a peak at all; assets with thin history (like 662 above) get a suggestion but it should be treated as lower-confidence until more data accumulates.
- **Preload measurement is opportunistic.** No-credit events aren't guaranteed to occur in every window; when none are seen, we assume 0 and say so explicitly rather than hiding the gap.
- **The 20 L target is a business assumption, not a measurement.** The model assumes the fleet's standard transaction is intended to deliver 20 L; if a site's authorized fill size differs, the formula and target should be revisited.
- **No device is changed without a human clicking Apply** after reviewing the exact before/after numbers.